

Week 13: Attention, Transformers and Generative AI

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-13/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-13/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-13/lab.html>

Colab notebook with Python step 13 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-13/NB13_transformers_llms_genai.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Engineering knowledge lives in text: standards, specifications, maintenance logs, test reports and code. Large language models process such text with the transformer architecture, whose self-attention lets every token weigh every other token [1]. Pretrained on vast corpora and tuned to follow instructions, they draft, summarise, translate and write code [2, 3, 4], but they also produce fluent statements that are false [5]. This week builds tokenisation, attention and a tiny transformer from scratch, connects a language task to retrieval so that answers rest on cited documents [6], surveys generative models for images and designs [7, 8], and sets out rules for responsible use in engineering work.

Estimated study time: 10 to 12 hours.

Final project. The final project is due at the end of Week 14. The [project page](#) lists the deliverables and the evaluation criteria.

Learning outcomes

By the end of the week, students are expected to explain tokenisation, embeddings and scaled dot-product attention, to compute attention weights for a small example, to describe how decoder-only language models are pretrained and tuned, to explain the effect of sampling temperature, to build and evaluate a retrieval step that grounds answers in documents, to name the main families of generative models, and to state the verification and confidentiality duties that apply when engineers use these tools.

Python in this week

These notes close with Python step 13: Text, vectors and attention. It covers strings and their methods, counting words with Counter, regular expressions for quantities with units, JSON files, bag-of-words

vectors with cosine similarity for retrieval, and softmax attention with a causal mask, applied to maintenance logs, specifications and word embeddings [1, 9]. In the course materials, the step is taught at the start of the Colab notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

Text as engineering data

A large share of engineering knowledge is written: standards and codes, design reports, maintenance and incident logs, requirements, patents and source code. Natural language processing turns such text into data. Classic tasks include search and retrieval, classification of reports, extraction of quantities and entities, summarisation and translation. Information retrieval developed many of the tools, such as term weighting and vector space models, long before neural networks [10]. Large language models add the ability to generate text and code, which changes how engineers draft, search and program, and raises new questions about accuracy and responsibility.

Formulas: Term weighting and cosine similarity

A vector space model represents a document d by one number per word t of the vocabulary. The term frequency $\text{tf}(t, d)$ counts the occurrences of the word in the document. The inverse document frequency rewards words that occur in few of the N documents, of which $\text{df}(t)$ contain the word [10]:

$$\text{idf}(t) = \ln \frac{N}{\text{df}(t)}, w(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

The library scikit-learn, used in the notebook, smooths the formula by adding 1 to both counts and to the logarithm, $\text{idf}(t) = \ln \frac{1+N}{1+\text{df}(t)} + 1$, so that no weight becomes zero. The notebook also uses sublinear term frequencies, $1 + \ln \text{tf}$ instead of tf , which damp words repeated within one record; for a word that occurs once, both give 1. A query is weighted in the same way, and documents are ranked by the cosine of the angle between their vectors and the query vector:

$$\cos(q, d) = \frac{q \cdot d}{\|q\| \|d\|} = \frac{\sum_t q_t d_t}{\sqrt{\sum_t q_t^2} \sqrt{\sum_t d_t^2}}$$

When all vectors are first divided by their lengths, the cosine is simply the dot product of the normalised vectors. It is 1 for the same direction and 0 when the texts share no word.

Worked example: Ranking maintenance records for a query

Three maintenance records contain, after removing common words, the terms D1: pump, bearing, noise; D2: pump, seal, leak; and D3: valve, seal, replaced. An engineer searches for “seal leak”.

Step 1. Inverse document frequencies. With $N = 3$, the words pump and seal occur in two records and all other words in one. With the smoothed formula, a word in one record gets

$$\text{idf} = \ln \frac{1+3}{1+1} + 1 = \ln \frac{4}{2} + 1 = 0.693147 + 1 = 1.693147$$

and a word in two records gets

$$\text{idf} = \ln \frac{1+3}{1+2} + 1 = \ln \frac{4}{3} + 1 = 0.287682 + 1 = 1.287682$$

Every word occurs once per record, so its weight equals its idf.

Step 2. Lengths and normalised vectors. The squared length of a vector is the sum of its squared weights, and each normalised weight is a weight divided by the length. The query has the weights 1.287682 for seal and 1.693147 for leak:

$$\|q\|^2 = 1.287682^2 + 1.693147^2$$

$$\|q\|^2 = 1.658125 + 2.866747 = 4.524872$$

$$\|q\| = \sqrt{4.524872} = 2.127175$$

$$q_{\text{seal}} = \frac{1.287682}{2.127175} = 0.605348, q_{\text{leak}} = \frac{1.693147}{2.127175} = 0.79596$$

The record D1 has the weights 1.287682 for pump and 1.693147 for bearing and noise:

$$\|d_1\|^2 = 1.287682^2 + 1.693147^2 + 1.693147^2$$

$$\|d_1\|^2 = 1.658125 + 2.866747 + 2.866747 = 7.391619$$

$$\|d_1\| = \sqrt{7.391619} = 2.718753$$

$$d_{1,\text{pump}} = \frac{1.287682}{2.718753} = 0.47363, d_{1,\text{bearing}} = d_{1,\text{noise}} = \frac{1.693147}{2.718753} = 0.622766$$

The record D2 has the weights 1.287682 for pump and seal and 1.693147 for leak:

$$\|d_2\|^2 = 1.287682^2 + 1.287682^2 + 1.693147^2$$

$$\|d_2\|^2 = 1.658125 + 1.658125 + 2.866747 = 6.182997$$

$$\|d_2\| = \sqrt{6.182997} = 2.486563$$

$$d_{2,\text{pump}} = d_{2,\text{seal}} = \frac{1.287682}{2.486563} = 0.517856, d_{2,\text{leak}} = \frac{1.693147}{2.486563} = 0.680919$$

The record D3 has the weights 1.693147 for valve and replaced and 1.287682 for seal:

$$\|d_3\|^2 = 1.693147^2 + 1.287682^2 + 1.693147^2$$

$$\|d_3\|^2 = 2.866747 + 1.658125 + 2.866747 = 7.391619$$

$$\|d_3\| = \sqrt{7.391619} = 2.718753$$

$$d_{3,\text{valve}} = d_{3,\text{replaced}} = \frac{1.693147}{2.718753} = 0.622766, d_{3,\text{seal}} = \frac{1.287682}{2.718753} = 0.47363$$

Step 3. Cosine similarities. Only shared words contribute:

$$\cos(q, D1) = 0$$

$$\cos(q, D2) = 0.605348 \times 0.517856 + 0.79596 \times 0.680919$$

$$\cos(q, D2) = 0.313483 + 0.541984 = 0.855467$$

$$\cos(q, D3) = 0.605348 \times 0.47363 = 0.286711$$

The ranking is D2, D3, D1. D3 shares only the common word seal, while D2 also contains the rarer word leak, whose larger idf gives it more weight. Part A of the [interactive lab](#) computes these values for any records and query.

From words to vectors

A model first splits text into tokens. Whole words give huge vocabularies and fail on new words; single characters make sequences long. Byte-pair encoding starts from characters and repeatedly merges the most frequent adjacent pair into a new symbol, so that common words become single tokens while rare words split into meaningful pieces [11]. Each token is then mapped to a vector, its embedding. Word2vec showed that embeddings learned from co-occurrence capture semantic relations, so that words used in similar contexts receive similar vectors [12]. Sentence embeddings extend the idea to whole passages and support semantic search [13].

Formulas: Byte-pair merges, embeddings and the skip-gram model

Byte-pair encoding writes every word of a training corpus as a sequence of characters with an end marker, here `_`, and counts every pair of adjacent symbols, each occurrence weighted by the frequency of its word. The most frequent pair is merged into a new symbol everywhere, and the counting repeats until the vocabulary has the desired size [11]. A new text is split by applying the learned merges in the same order.

Each token i of a vocabulary of size V owns a row e_i of an embedding matrix E with V rows and d columns. Multiplying the one-hot vector of the token, which has a 1 at position i and 0 elsewhere, by E selects this row, so an embedding layer is a table lookup whose entries are learned.

The skip-gram model of word2vec [12] learns two vectors per word, v_w for the word in the centre of a window and u_w for the word as context. The probability that the word o appears near the centre word c is modelled as a softmax over the vocabulary:

$$p(o|c) = \frac{e^{u_o \cdot v_c}}{\sum_{w=1}^V e^{u_w \cdot v_c}}$$

Training raises these probabilities for the word pairs observed in text, so that words with similar contexts end up with similar vectors.

Worked example: Byte-pair merges and context probabilities

Step 1. The corpus. Maintenance logs contain the word pump 5 times, pumps 3 times, dump 2 times and pumped 2 times, written as `p u m p _`, `p u m p s _`, `d u m p _` and `p u m p e d _`.

Step 2. The first merge. The pair `u m` occurs once in each word, so it counts $5+3+2+2=12$; `m p` also counts 12, `p u` counts $5+3+2=10$, and `p _` counts $5+2=7$, from pump and dump. Of the two pairs with 12, `u m` appears first and is merged; ties are broken by the order of first appearance.

Step 3. Four merges. Each merge changes the counts of the next round:

Merge	Most frequent pair: count	Next candidates	New symbol
1	<code>u m</code> : 12	<code>m p</code> : 12; <code>p u</code> : 10	<code>um</code>
2	<code>um p</code> : 12	<code>p um</code> : 10; <code>p _</code> : 7	<code>ump</code>
3	<code>p ump</code> : 10	<code>ump _</code> : 7; <code>ump s</code> : 3	<code>pump</code>
4	<code>pump _</code> : 5	<code>pump s</code> : 3; <code>s _</code> : 3	<code>pump_</code>

After four merges, `pump_` is a single token, `pumps` splits into `pump`, `s` and `_`, `dump` into `d`, `ump` and `_`, and `pumped` into `pump`, `e`, `d` and `_`. The frequent word became one token, and the rare ones share its pieces.

Step 4. Context probabilities. The centre vector of bearing is $v_c = (0.5, 1)$, and three context vectors are $u = (1, 0.8)$ for temperature, $(0.6, 1.2)$ for lubricant and $(-0.4, 0.1)$ for concrete. If the softmax runs over only these three context words:

$$u \cdot v_c = 0.5 + 0.8 = 1.3, 0.3 + 1.2 = 1.5, -0.2 + 0.1 = -0.1$$

$$e^{1.3} = 3.669297, e^{1.5} = 4.481689, e^{-0.1} = 0.904837, \text{sum} = 9.055823$$

$$p = \frac{3.669297}{9.055823} = 0.405186, \frac{4.481689}{9.055823} = 0.494896, \frac{0.904837}{9.055823} = 0.099918$$

The model expects temperature and lubricant near bearing and finds concrete unlikely. Part B of the lab learns byte-pair merges from a larger corpus.

Attention and the transformer

Attention was introduced to let a translation model look at the relevant words of the source sentence while producing each output word [14]. The transformer made attention the central operation [1]. Every token produces a query, a key and a value vector through learned linear maps. The attention weights of a token are the softmax of the dot products of its query with all keys, divided by the square root of the key dimension; its output is the weighted sum of the values. Several attention heads run in parallel and learn different relations, positional encodings inject word order, and residual connections and layer normalisation, as in the ResNets of Week 11, keep deep stacks trainable.

Two families dominate. Encoder models such as BERT see the whole text at once and are pretrained to fill in masked words, which suits classification and extraction [15]. Decoder models such as GPT are pretrained to predict the next token from the previous ones, with a causal mask that hides the future, and generate text one token at a time [2, 16]. Transformers now process images, sequences of measurements and molecules as well [17, 18].

Animation: Where does each token look?. Select a token of the sentence to see its attention weights over the others. The embeddings here are small hand-made vectors, chosen for illustration rather than learned; the temperature scales the dot products before the softmax. Run it on the [lecture page](#).

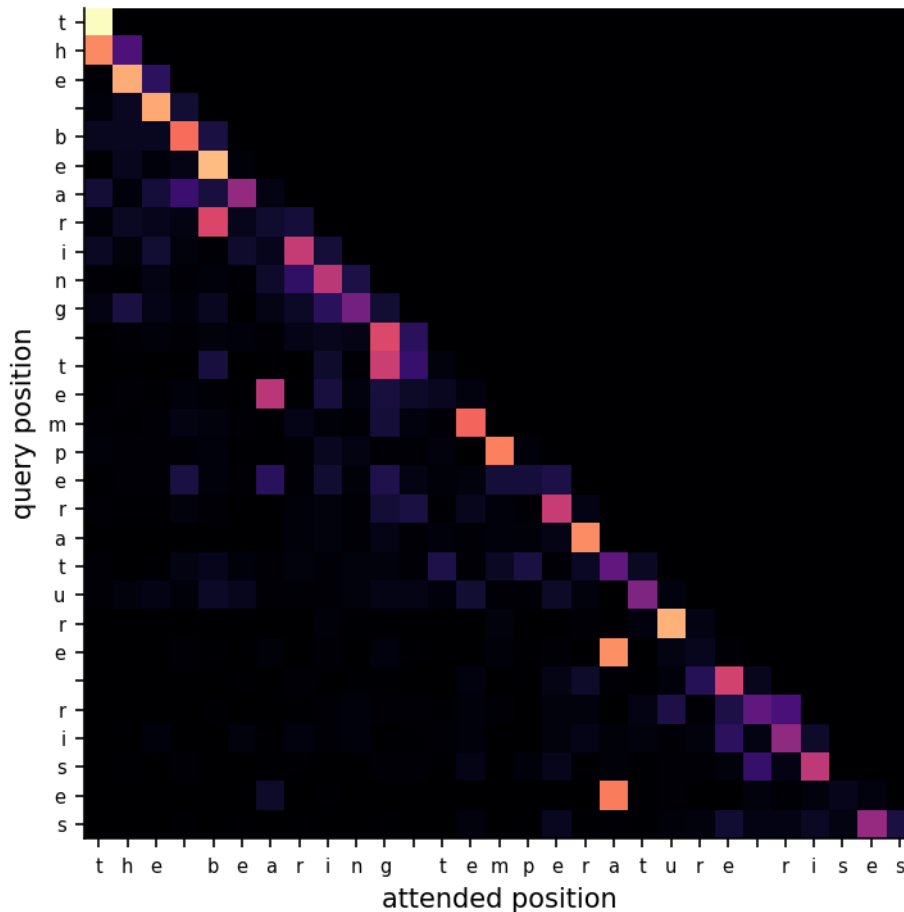


Figure 13.1. Attention weights of one head of the tiny character-level transformer trained in the notebook. The causal mask leaves the upper triangle empty: No position may look at later characters.

Check your understanding. In scaled dot-product attention, why are the dot products divided by the square root of the key dimension?

- A. To make the weights sum to one
- B. To keep the dot products from growing with the dimension, which would push the softmax into saturation
- C. To hide future tokens
- D. To reduce the number of parameters

Answer: B. Large dot products make the softmax nearly one-hot and its gradients tiny; the scaling keeps them in a useful range [1].

Formulas: Scaled dot-product attention, heads and positions

The n tokens of a sequence enter a transformer layer as the rows of a matrix X with n rows and d columns. Learned matrices map every token to a query, a key and a value [1]:

$$Q = X W_Q, K = X W_K, V = X W_V$$

The attention weights of token i are the softmax of the dot products of its query with all keys, divided by the square root of the key length d_k , and its output is the weighted sum of the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$

$$s_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}, a_{ij} = \frac{e^{s_{ij}}}{\sum_m e^{s_{im}}}$$

The softmax acts on every row of the scores. The division keeps the scores in a useful range: For queries and keys whose components are independent with mean 0 and variance 1, the dot product

$q \cdot k = \sum_{a=1}^{d_k} q_a k_a$ has the variance d_k , and large scores would push the softmax towards a single weight of 1, where its gradients vanish. A decoder that generates text adds a causal mask, which sets $s_{ij} = -\infty$ for $j > i$, so that no token sees later tokens and those weights become 0.

Multi-head attention runs h such computations with their own smaller matrices in parallel, joins their outputs and maps them back with a matrix W_O . Because attention treats its inputs as a set, positional encodings are added to the embeddings. The original transformer uses waves of different wavelengths, for the position pos and the dimensions $2i$ and $2i+1$ of a model of width d :

$$\text{PE}(\text{pos}, 2i) = \sin \frac{\text{pos}}{10000^{2i/d}}, \text{PE}(\text{pos}, 2i+1) = \cos \frac{\text{pos}}{10000^{2i/d}}$$

Worked example: Attention over three tokens

The tokens pump, seal and leaks have two-dimensional embeddings, and the layer has the matrices

$$W_Q = (1, 0.5; 0, 1), W_K = (1, 0; 0.5, 1), W_V = (2, 0; 0, 1)$$

written row by row, with the rows separated by a semicolon.

Step 1. Queries, keys and values. Each row of X times a matrix gives the vectors of a token; for example, the query of leaks is $(1, 1)$ times W_Q , that is $(1 \times 1 + 1 \times 0, 1 \times 0.5 + 1 \times 1) = (1, 1.5)$:

Token	x	q	k	v
1, pump	$(1, 0)$	$(1, 0.5)$	$(1, 0)$	$(2, 0)$
2, seal	$(0, 1)$	$(0, 1)$	$(0.5, 1)$	$(0, 1)$
3, leaks	$(1, 1)$	$(1, 1.5)$	$(1.5, 1)$	$(2, 1)$

Step 2. Scaled scores. The dot products $q_i \cdot k_j$, for example $q_3 \cdot k_3 = 1 \times 1.5 + 1.5 \times 1 = 3$, divided by $\sqrt{2} = 1.414214$:

Query	Key 1	Key 2	Key 3
1	$1/1.414214 = 0.707107$	$1/1.414214 = 0.707107$	$2/1.414214 = 1.414213$
2	$0/1.414214 = 0$	$1/1.414214 = 0.707107$	$1/1.414214 = 0.707107$
3	$1/1.414214 = 0.707107$	$2/1.414214 = 1.414213$	$3/1.414214 = 2.12132$

Step 3. Softmax. Each row is exponentiated and divided by its sum:

Query	$e^{s_{i1}}, e^{s_{i2}}, e^{s_{i3}}$	Sum	Weights a_{i1}, a_{i2}, a_{i3}
1	2.028115, 2.028115, 4.113248	8.169478	0.248255, 0.248255, 0.50349
2	1, 2.028115, 2.028115	5.05623	0.197776, 0.401112, 0.401112
3	2.028115, 4.113248, 8.342142	14.483505	0.140029, 0.283995, 0.575975

Step 4. Outputs. Each output is the weighted sum of the values:

$$o_1 = 0.248255 \times (2, 0) + 0.248255 \times (0, 1) + 0.50349 \times (2, 1)$$

$$\begin{aligned}
o_1 &= (1.50349, 0.751745) \\
o_2 &= 0.197776 \times (2, 0) + 0.401112 \times (0, 1) + 0.401112 \times (2, 1) \\
o_2 &= (1.197776, 0.802224) \\
o_3 &= 0.140029 \times (2, 0) + 0.283995 \times (0, 1) + 0.575975 \times (2, 1) \\
o_3 &= (1.432008, 0.85997)
\end{aligned}$$

The token leaks attends most to itself, whose key gives the largest dot product with its query, and next to seal.

Step 5. The causal mask. In a decoder, seal may only look at pump and itself. Its scores 0 and 0.707107 give $e^0 = 1$ and $e^{0.707107} = 2.028115$, the sum 3.028115 and the weights 0.330238 and 0.669762, so

$$o_2 = 0.330238 \times (2, 0) + 0.669762 \times (0, 1) = (0.660476, 0.669762)$$

The first token can only attend to itself and outputs its own value, $(2, 0)$. Part C of the [interactive lab](#) computes attention for any tokens and matrices.

Worked example: Positions and the scaling factor

Step 1. Positional encodings. For a model of width $d = 4$, the dimensions 0 and 1 use $10000^{0/4} = 1$ and the dimensions 2 and 3 use $10000^{2/4} = 100$:

Position	$\sin(\text{pos})$	$\cos(\text{pos})$	$\sin(\text{pos}/100)$	$\cos(\text{pos}/100)$
0	0	1	0	1
1	0.841471	0.540302	0.01	0.99995
2	0.909297	-0.416147	0.019999	0.9998

The fast waves distinguish neighbouring positions, and the slow waves keep distant positions apart.

Step 2. Why divide by the square root. With $d_k = 64$ and components of variance 1, the dot products have the variance 64 and the standard deviation $\sqrt{64} = 8$. Scores of 16, 8 and 0, that is two, one and zero standard deviations above the mean of 0, give

$$\text{softmax}(16, 8, 0) = (0.999665, 0.000335, 0)$$

with $e^{16} = 8,886,110.52$, $e^8 = 2980.96$ and $e^0 = 1$, an almost one-hot distribution. Divided by 8, the same scores become 2, 1 and 0 and give

$$\text{softmax}(2, 1, 0) = \left(\frac{7.389056}{11.107338}, \frac{2.718282}{11.107338}, \frac{1}{11.107338} \right) = (0.665241, 0.244728, 0.090031)$$

a distribution that still spreads its weight, so the gradients do not vanish and the model can keep learning. Part D of the lab shows the encodings and the effect of the scaling.

Large language models

A language model assigns probabilities to the next token. Trained on hundreds of billions of tokens, decoder transformers develop broad abilities, and performance improves predictably with model size, data and compute, as empirical scaling laws describe [19]. GPT-3 showed that a large model can perform new tasks from a few examples in the prompt, without changing its weights [2]. Instruction tuning and reinforcement learning from human feedback then align models with what users ask for [3]. Prompts that ask for intermediate reasoning steps improve performance on multi-step problems [20], and models

trained on code write and explain programs [4]. Such broadly trained models, adapted to many tasks, are called foundation models [21].

Generation samples tokens from the predicted distribution. A temperature below one sharpens the distribution towards the most likely tokens; a temperature above one flattens it and increases variety and the risk of nonsense. The model has no built-in notion of truth: It produces what is probable given its training, and fluent but unsupported statements, called hallucinations, are a known failure mode [5]. For an engineer, a model's answer about a load factor or a material property is a hypothesis to check, never a source.

Check your understanding. A model's next-token logits are 2.0, 1.0 and 0.0. What happens to the probability of the first token when the temperature is lowered from 1.0 to 0.5?

- A. It decreases
- B. It increases, because dividing the logits by a smaller temperature enlarges their differences
- C. It stays the same
- D. It becomes exactly 0.5

Answer: B. Dividing by 0.5 doubles the gaps between logits, so the softmax concentrates more mass on the largest one.

Formulas: Next-token probabilities, temperature, perplexity and scaling

A language model computes for the next token a score z_k , the logit, for every token k of the vocabulary, and turns the scores into probabilities with a softmax [16]. A temperature T divides the scores first:

$$p_k = \frac{e^{z_k/T}}{\sum_m e^{z_m/T}}$$

$T=1$ gives the trained distribution, $T<1$ sharpens it towards the most likely tokens, and $T>1$ flattens it. Top- k sampling keeps only the k tokens with the largest scores and divides their probabilities by their sum. Training minimises the cross-entropy, the mean negative logarithm of the probability that the model gave to each actual next token x_t of the training text, and the perplexity is its exponential:

$$\text{CE} = -\frac{1}{n} \sum_{t=1}^n \ln p(x_t | x_1, \dots, x_{t-1}), \text{PP} = e^{\text{CE}}$$

A perplexity of m means that the model is, on average, as uncertain as a choice among m equally likely tokens. Kaplan and colleagues found that the test loss falls as a power of the number N of parameters when data and compute are not limiting [19]:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \alpha_N \approx 0.076$$

so that every doubling of the model multiplies the loss by $2^{-0.076}$.

Worked example: Temperature and top-k for the next word

After "The concrete cube is tested after 28", a model gives the scores of Part E of the lab to eight candidate words:

Word	z	e^z	p at $T=1$	$z/0.5$	$e^{z/0.5}$	p at $T=0.5$
days	4.2	66.686331	0.835229	8.4	4447.066748	0.991015
hours	1.6	4.953032	0.062035	3.2	24.53253	0.005467
months	1.1	3.004166	0.037626	2.2	9.025013	0.002011
cubes	0.2	1.221403	0.015298	0.4	1.491825	0.000332

Word	z	e^z	p at $T=1$	$z/0.5$	$e^{z/0.5}$	p at $T=0.5$
MPa	0.4	1.491825	0.018685	0.8	2.225541	0.000496
years	0.3	1.349859	0.016907	0.6	1.822119	0.000406
minutes	0.1	1.105171	0.013842	0.2	1.221403	0.000272
bananas	-3.5	0.030197	0.000378	-7	0.000912	0

Step 1. Temperature 1. The exponentials add up to 79.841984, so days has the probability $66.686331/79.841984 = 0.835229$.

Step 2. Temperature 0.5. The doubled scores give the sum 4487.386091, and days rises to $4447.066748/4487.386091 = 0.991015$; nonsense such as bananas falls to practically zero.

Step 3. Top-3 at $T=1$. Only days, hours and months remain. Their exponentials add up to $66.686331 + 4.953032 + 3.004166 = 74.643529$, so their probabilities become 0.893397, 0.066356 and 0.040247.

Worked example: Perplexity and scaling

Step 1. Cross-entropy. A model gives the four actual next tokens of a short text the probabilities 0.5, 0.25, 0.8 and 0.1:

$$-\ln 0.5 = 0.693147, -\ln 0.25 = 1.386294, -\ln 0.8 = 0.223144, -\ln 0.1 = 2.302585$$

$$\text{CE} = \frac{0.693147 + 1.386294 + 0.223144 + 2.302585}{4} = 1.151293$$

Step 2. Perplexity. $\text{PP} = e^{1.151293} = 3.162279$. The product of the four probabilities is 0.01, and $0.01^{-1/4} = \sqrt[4]{10} = 3.162278$ is the same value up to the rounding of the cross-entropy: The model was as uncertain as a choice among about three tokens.

Step 3. Scaling. By the scaling law, doubling the parameters multiplies the loss by $2^{-0.076} = 0.948684$, a reduction of about 5%, and ten times as many parameters multiply it by $10^{-0.076} = 0.83946$. Large gains need very large increases of size, data and compute. Part F of the [interactive lab](#) computes these quantities.

Retrieval-augmented generation

Retrieval-augmented generation combines a retriever with a generator [6]. The retriever searches a document collection, for example a company's standards or a project's reports, for passages relevant to the question. The generator then answers using those passages and can cite them. Grounding reduces hallucination, keeps answers current without retraining, and lets a reader verify each claim against its source. Retrieval can use sparse term weighting such as TF-IDF [10] or dense sentence embeddings [13]. Its quality can be measured like any classifier: For a set of questions with known relevant passages, the share of questions whose relevant passage appears among the top results is the hit rate.

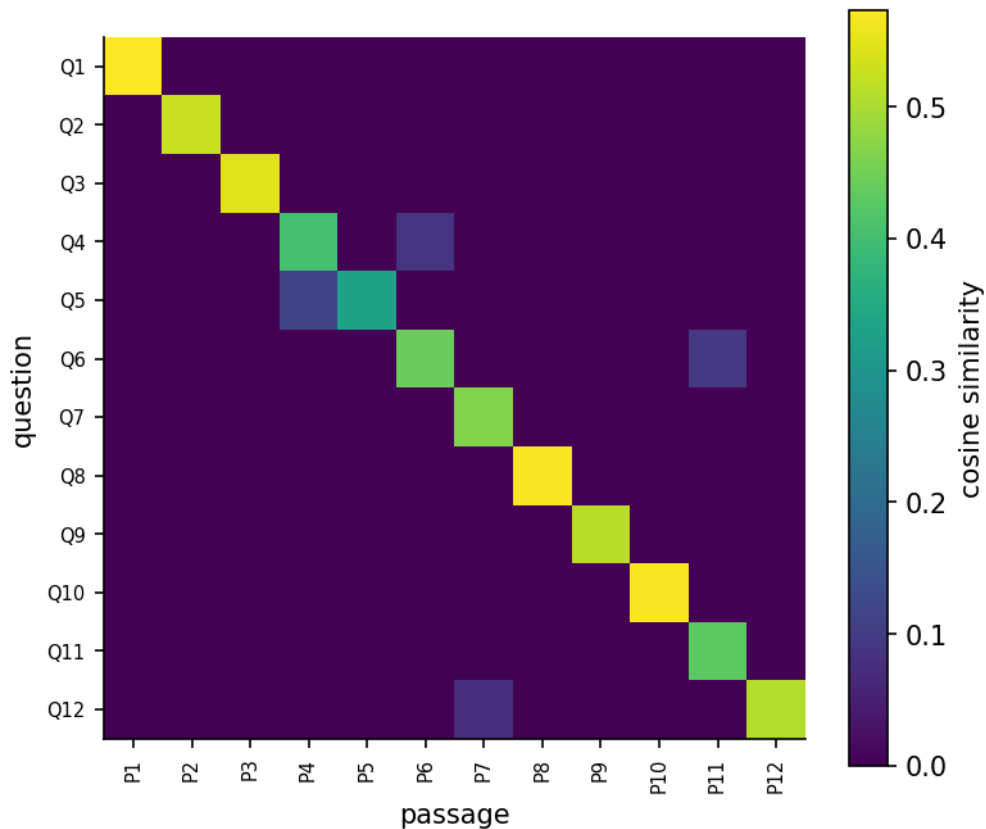


Figure 13.2. Cosine similarities between test questions and the passages of the notebook's small engineering knowledge base, computed with TF-IDF vectors. Bright cells on the diagonal mean the right passage is found first.

Check your understanding. What is the main engineering benefit of retrieval-augmented generation over asking a model directly?

- A. It removes the need for any verification
- B. Answers are grounded in cited documents that can be checked and updated
- C. It makes the model smaller
- D. It guarantees correct answers

Answer: B. Retrieval ties answers to sources; checking them remains necessary, but it becomes possible [6].

Formulas: Retrieval scores and their evaluation

A dense retriever encodes the question and every passage into vectors and ranks the passages by the cosines between these vectors [13]; for vectors of length 1, the cosine is simply the dot product. The generator then conditions its answer on the k best passages [6]. Retrieval is evaluated on test questions whose relevant passages are known [10]. If the relevant passages of Q questions appear at the ranks $r_1, \dots, r_Q$, the hit rate at k is the share of questions whose relevant passage is among the first k , and the mean reciprocal rank rewards passages found at a high place:

$$\text{hit}@k = \frac{1}{Q} \sum_{i=1}^Q [r_i \leq k], \text{MRR} = \frac{1}{Q} \sum_{i=1}^Q \frac{1}{r_i}$$

where $[r_i \leq k]$ is 1 when the condition holds and 0 otherwise. The prompt must also fit the context window of the generator: The tokens of the instructions, the question and the k passages add up and must stay below the limit.

Worked example: Retrieving and scoring passages

Step 1. Dense retrieval. A question and three passages have the normalised embeddings $q=(0.6, 0.8, 0)$, $P1=(0.8, 0.6, 0)$, $P2=(0, 0.6, 0.8)$ and $P3=(0.6, 0, 0.8)$:

$$q \cdot P1 = 0.6 \times 0.8 + 0.8 \times 0.6 + 0 \times 0 = 0.96$$

$$q \cdot P2 = 0 + 0.8 \times 0.6 + 0 = 0.48, q \cdot P3 = 0.6 \times 0.6 + 0 + 0 = 0.36$$

The ranking is P1, P2, P3, and with $k=2$ the generator receives P1 and P2.

Step 2. Evaluation. For five test questions, the relevant passages are found at the ranks 1, 2, 1, 4 and 1:

$$\text{hit@1} = \frac{3}{5} = 0.6, \text{hit@3} = \frac{4}{5} = 0.8$$

$$\text{MRR} = \frac{1 + 1/2 + 1 + 1/4 + 1}{5} = \frac{3.75}{5} = 0.75$$

The fourth question needs attention: Its passage appears only at rank 4, so a generator given three passages would answer it without the evidence.

Step 3. The context budget. Instructions of 50 tokens, a question of 20 tokens and three passages of about 60 tokens each need $50 + 20 + 3 \times 60 = 250$ tokens, which fits a context window of 512 tokens with room for the answer. Part G of the [interactive lab](#) retrieves passages with citations, and Part H repeats these three steps for other embeddings, ranks and budgets.

Generative models beyond text

Variational autoencoders learn a latent space from which new samples can be decoded [22]. Generative adversarial networks train a generator against a discriminator that tries to tell real from generated data [23]. Diffusion models learn to reverse a gradual noising process and now produce high-quality images; latent diffusion runs the process in a compressed space to make it efficient [7, 8]. In engineering, generative models propose designs, create synthetic training images for rare defects, and suggest candidate materials; deep learning has proposed hundreds of thousands of stable crystal structures [24], and language models have planned and run chemistry experiments with robotic equipment [25]. Every generated design still has to satisfy the physics, the codes and the tests.

Formulas: Variational autoencoders, adversarial networks and diffusion

A variational autoencoder encodes an input x into the mean μ and the standard deviation σ of a latent variable z , draws $z = \mu + \sigma \varepsilon$ with ε from the standard normal distribution, which lets gradients pass through the sampling, and decodes z [22]. Training maximises a lower bound of the log-likelihood, a reconstruction term minus the Kullback-Leibler divergence of the latent distribution from the standard normal, which for one latent dimension is

$$\text{KL} = \frac{1}{2} (\mu^2 + \sigma^2 - 1 - \ln \sigma^2)$$

A generative adversarial network trains a generator G , which turns noise z into samples, against a discriminator D , which estimates the probability that its input is real [23]:

$$\min_G \max_D E_x [\ln D(x)] + E_z [\ln (1 - D(G(z)))]$$

For a fixed generator with the sample density p_g , the best discriminator is

$D^*(x) = p_{\text{data}}(x) / (p_{\text{data}}(x) + p_g(x))$. When the generator matches the data, $D^* = 1/2$ everywhere and the value of the game is $-\ln 4$.

A diffusion model destroys data step by step with Gaussian noise of variances $\beta_1, \dots, \beta_T$ and learns to reverse the process [7]. Any step can be reached from the clean sample x_0 directly:

$$\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s), x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$$

The network learns to predict the noise ε from x_t and t , and generation runs the steps backwards from pure noise. Ho and colleagues used $T = 1000$ steps with β rising linearly from 10^{-4} to 0.02.

Worked example: Three generative models by the numbers

Step 1. Diffusion. A pixel with the value $x_0 = 0.8$, on the scale from -1 to 1 , passes three steps with the illustrative variances 0.1, 0.2 and 0.3:

$$\bar{\alpha}_1 = 0.9, \bar{\alpha}_2 = 0.9 \times 0.8 = 0.72, \bar{\alpha}_3 = 0.72 \times 0.7 = 0.504$$

With the noise draw $\varepsilon = -0.5$:

$$x_3 = \sqrt{0.504} \times 0.8 + \sqrt{0.496} \times (-0.5) = 0.70993 \times 0.8 - 0.704273 \times 0.5$$

$$x_3 = 0.567944 - 0.352137 = 0.215807$$

With the schedule of Ho and colleagues, the product of all 1000 factors is about 4.04×10^{-5} , so $\sqrt{\bar{\alpha}_{1000}} \approx 0.0064$, and less than 1% of the original signal remains.

Step 2. The latent cost of a VAE. For $\mu = 0.5$ and $\sigma = 0.8$, $\ln \sigma^2 = \ln 0.64 = -0.446287$, and

$$\text{KL} = \frac{1}{2} (0.25 + 0.64 - 1 - (-0.446287)) = \frac{1}{2} \times 0.336287 = 0.168144$$

A draw $\varepsilon = -1.2$ gives the latent value $z = 0.5 + 0.8 \times (-1.2) = -0.46$.

Step 3. The best discriminator. Where the data have the density 0.3 and the generator 0.1, $D^* = 0.3 / (0.3 + 0.1) = 0.75$: The point looks real. Once the generator has learned the data distribution, the two densities agree everywhere and $D^* = 1/2$. Part I of the [interactive lab](#) runs the forward diffusion of a small image and these calculations.

Responsible use in engineering work

Engineers remain accountable for their work, whatever tools they use. Four rules follow. Verify every factual or numerical claim against a primary source, a calculation or a test. Do not paste confidential designs, personal data or unpublished results into external services unless the organisation's policy allows it. Cite the sources that support the work, not the tool. And treat model inputs as an attack surface: Instructions hidden in documents can manipulate a model that reads them, and small perturbations can change the decisions of learned models [26]. The energy cost of training and running large models is also part of the engineering trade-off [27]. The course policy on the use of these tools is stated in the syllabus.

Formulas: Energy and emissions of computation

The electrical energy of a computation is the power of the hardware times the running time. Data centres need additional energy for cooling and power supply, which the power usage effectiveness PUE accounts for, and the carbon intensity c of the electricity converts energy into emissions [27]:

$$E = \text{PUE} \times \sum_{\text{devices}} P \times t, m_{\text{CO}_2} = c E$$

Strubell and colleagues used a PUE of 1.58 and the average carbon intensity of the U.S. grid, 0.954 pounds, about 0.433 kg, of carbon dioxide per kWh. Both values vary strongly between data centres and countries, so an estimate should state its assumptions.

Worked example: The footprint of a fine-tuning run

A model is fine-tuned on 8 graphics processors that draw 300 W each for 72 hours:

$$8 \times 300 \text{ W} \times 72 \text{ h} = 172,800 \text{ Wh} = 172.8 \text{ kWh}$$

$$E = 1.58 \times 172.8 = 273.024 \text{ kWh}, m_{\text{CO}_2} = 0.433 \times 273.024 = 118.2194 \text{ kg}$$

Repeating the run for every hyperparameter setting multiplies these numbers, which is one reason to plan experiments carefully.

Python step 13: Text, vectors and attention

Strings

Text is a sequence of characters. Strings support indexing and slicing like lists, and their methods return transformed copies: `strip` removes surrounding spaces, `lower` converts to lower case, `split` breaks a string into a list of words, and `join` glues a list of strings together. `in` tests whether a piece of text occurs, and `count` counts its occurrences. Strings cannot be changed in place, so every method returns a new string.

```
line = " 2026-03-14 PUMP-3: High vibration at bearing DE; replaced  
seal.  "  
clean = line.strip()  
print(clean[:10], "|", clean[11:17])  
print(clean.lower().split())  
print("vibration" in clean.lower(), clean.count("e"))  
print(" / ".join(["seal", "bearing", "impeller"]))  
print(len("İSTANBUL"), len("İSTANBUL".lower()))
```

Output

```
2026-03-14 | PUMP-3  
['2026-03-14', 'pump-3:', 'high', 'vibration', 'at', 'bearing', 'de;',  
'replaced', 'seal.']  
True 4  
seal / bearing / impeller  
8 9
```

The last line shows a detail that matters for Turkish text. Converting the capital dotted İ to lower case with the default Unicode rules produces an i followed by a combining dot, so the lower-case string is one character longer than the original. Language-aware processing, for example search in Turkish documents, has to treat such letters explicitly.

Counting words

`collections.Counter` counts the elements of any sequence and reports the most common ones. The nested comprehension below turns several log entries into one list of words: The first `for` runs over the entries and the second over the words of each entry. `re.findall(r"[a-z]+", ...)` keeps runs of letters and drops digits and punctuation. A dictionary comprehension builds a dictionary in the same compact way, and `sorted` with a `key` orders items by a computed value.

```
import re  
from collections import Counter
```

```
logs = ["High vibration at pump 3, bearing replaced.",
        "Pump 3 seal leaking; seal replaced.",
        "Low flow at pump 1, strainer cleaned.",
        "Vibration normal after bearing replacement at pump 3."]
words = [w for entry in logs for w in re.findall(r"[a-z]+",
entry.lower())]
counts = Counter(words)
print(counts.most_common(4))
repeated = {w: n for w, n in counts.items() if n > 1}
print(sorted(repeated, key=repeated.get, reverse=True))
```

Output

```
[('pump', 4), ('at', 3), ('vibration', 2), ('bearing', 2)]
['pump', 'at', 'vibration', 'bearing', 'replaced', 'seal']
```

Word counts of this kind are the starting point of the TF-IDF retrieval of this week, which weights each word by how rare it is across all documents.

Patterns with regular expressions

Regular expressions describe patterns of text. `\d+` matches one or more digits, `(?:\.\d+)?` an optional decimal part, `\s*` optional spaces and `(MPa|kN|mm)` one of three units. Parentheses without `?:` capture the parts that `re.findall` returns, here pairs of value and unit taken from a specification sentence.

```
spec = "Use C30/37 concrete with 28-day strength 38 MPa, cover 40 mm
and a column load of 1250.5 kN."
pairs = re.findall(r"(\d+(?:\.\d+)?)\s*(MPa|kN|mm)", spec)
print(pairs)
print({unit: float(value) for value, unit in pairs})
```

Output

```
[('38', 'MPa'), ('40', 'mm'), ('1250.5', 'kN')]
{'MPa': 38.0, 'mm': 40.0, 'kN': 1250.5}
```

Saving and loading structured text

Files are opened with `open` inside a `with` block, which closes the file automatically. JSON stores lists and dictionaries as text that other programs and languages can read, a convenient format for a small knowledge base of passages such as the one used for retrieval in this week.

```
import json

passages = [{"id": "P1", "text": "Strength falls as the water-cement
ratio rises."},
            {"id": "P2", "text": "Clays plot above the A-line of the
plasticity chart."}]
with open("kb_demo.json", "w", encoding="utf-8") as f:
    json.dump(passages, f, indent=1)
with open("kb_demo.json", encoding="utf-8") as f:
    loaded = json.load(f)
print(len(loaded), loaded[1]["id"], loaded[1]["text"])
```

Output

```
2 P2 Clays plot above the A-line of the plasticity chart.
```

From counts to vectors: retrieval

Retrieval compares a question with stored passages. A bag-of-words vector counts how often each word of a fixed vocabulary occurs in a text. The cosine similarity of two vectors, their dot product divided by the product of their lengths, is 1 for texts with the same word proportions and 0 for texts without a common word. Stacking the vectors of all passages into a matrix turns the comparison with a question into one matrix-vector product. This is the retrieval step of retrieval-augmented generation in its simplest form; TF-IDF weighting refines it by giving rare words more influence.

```
import numpy as np
```

```
vocab = sorted(set(words))
col = {w: i for i, w in enumerate(vocab)}
```

```
def bow(text):
    """Bag-of-words count vector of a text over the vocabulary."""
    v = np.zeros(len(vocab))
    for w in re.findall(r"[a-z]+", text.lower()):
        if w in col:
            v[col[w]] += 1
    return v
```

```
D = np.array([bow(entry) for entry in logs])           # one row per log
entry
q = bow("bearing vibration")
sim = D @ q / (np.linalg.norm(D, axis=1) * np.linalg.norm(q))
print(D.shape, sim.round(2))
print("best match:", logs[int(np.argmax(sim))])
```

Output

```
(4, 15) [0.58 0.  0.  0.53]
best match: High vibration at pump 3, bearing replaced.
```

Softmax and attention

Attention lets every token of a sequence gather information from the other tokens [1]. Each token has a query, a key and a value vector. The scores are the dot products of queries and keys, divided by the square root of their length. A softmax along each row turns the scores into weights that are positive and sum to one, and each output is the weighted mean of the values. In a transformer, learned matrices produce queries, keys and values from the token embeddings; the toy example below uses three hand-made embeddings directly, with “pump” and “impeller” deliberately similar.

```
def softmax(s, axis=-1):
    """Softmax along an axis; subtracting the maximum avoids
    overflow."""
    e = np.exp(s - s.max(axis=axis, keepdims=True))
    return e / e.sum(axis=axis, keepdims=True)
```

```
emb = np.array([[1.0, 0.0, 1.0, 0.0],           # "pump"
                [0.0, 1.0, 0.0, 1.0],           # "seal"
                [1.0, 0.0, 0.9, 0.1]])          # "impeller"
Q, K, V = emb, emb, emb
scores = Q @ K.T / np.sqrt(emb.shape[1])
weights = softmax(scores, axis=1)
```



```
print(weights.round(2))
print((weights @ V).round(2))
```

Output

```
[[0.43 0.16 0.41]
 [0.21 0.57 0.22]
 [0.42 0.17 0.41]]
[[0.84 0.16 0.8 0.2 ]
 [0.43 0.57 0.41 0.59]
 [0.83 0.17 0.79 0.21]]
```

A model that generates text one token at a time must not look at future tokens. A causal mask sets the scores above the diagonal to minus infinity before the softmax, so that every token attends only to itself and to earlier tokens, as in the decoders of generative language models [2].

```
future = np.triu(np.ones((3, 3), dtype=bool), k=1)      # True above the
diagonal
print(softmax(np.where(future, -np.inf, scores), axis=1).round(2))
```

Output

```
[[1.  0.  0. ]
 [0.27 0.73 0. ]
 [0.42 0.17 0.41]]
```

Check your understanding. What does “pump seal leak”.split() return?

- A. [“pump”, “seal”, “leak”]
- B. “pumpsealleak”
- C. [“p”, “u”, “m”, “p”]
- D. 3

Answer: A. Without an argument, split breaks the string at runs of white space and returns the words as a list.

Discipline challenges

Every department writes and reads text. Choose a row and adapt the notebook’s retrieval section, or its tiny language model, to the documents named there.

Department	Challenge
Computer Engineering	Retrieval over the documentation of a software library; evaluate code suggestions against unit tests [4].
Civil Engineering	Retrieval over summaries of design code clauses written by the student; answer questions with clause citations.
Mechanical Engineering and Industrial Engineering	Classify maintenance log entries by failure mode with TF-IDF and a linear classifier.
Electrical and Electronics Engineering	Extract ratings and quantities from equipment datasheets and check them against the source.
Chemistry and Chemical Engineering	Summarise safety data sheet sections and verify every hazard statement against the original [25].
Environmental Engineering	Search environmental impact report passages for mitigation measures by topic.
Food Engineering and Biology	Retrieval over open protocols for laboratory

Department	Challenge
	methods with a check of every quantity.
Geological Engineering, Geophysical Engineering, Mining and Earth Sciences Engineering	Index field notes or borehole log descriptions and query lithologies.
Physics, Mathematics and Statistics	Implement multi-head attention in NumPy and verify it against PyTorch's <code>nn.MultiheadAttention</code> .
Textile Engineering and Automotive Engineering	Cluster customer complaint texts by topic and relate the clusters to process or design causes.

Weekly task

Build a small retrieval system for documents of your department that you are allowed to share, such as public standards summaries, course notes or open reports. Create at least ten passages and ten test questions with known answers, measure the hit rate at 1 and at 3 with TF-IDF, and discuss failure cases. If you also use a language model, show one grounded answer with citations and one answer that needed correction. Write about 500 words and cite at least three works from this week's references [1, 5, 6].

Research and report assignment (optional)

Language models in engineering practice. Review how large language models are evaluated or used in one engineering activity, such as code generation, requirements engineering, standards compliance checking, maintenance log analysis or laboratory automation. Discuss accuracy, verification, confidentiality and accountability, with attention to documented failure modes [4, 5, 21, 25].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (pp. 5998-6008). <https://arxiv.org/abs/1706.03762>
- [2] Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33* (pp. 1877-1901). <https://arxiv.org/abs/2005.14165>
- [3] Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35* (pp. 27730-27744). <https://arxiv.org/abs/2203.02155>
- [4] Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374. <https://arxiv.org/abs/2107.03374>
- [5] Ji, Z., Lee, N., Frieske, R., et al. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 248. <https://doi.org/10.1145/3571730>
- [6] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33* (pp. 9459-9474). <https://arxiv.org/abs/2005.11401>
- [7] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems 33* (pp. 6840-6851). <https://arxiv.org/abs/2006.11239>

- [8] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 10684-10695). <https://doi.org/10.1109/CVPR52688.2022.01042>
- [9] Python Software Foundation (2026). *The Python Tutorial (Python 3 documentation)*. <https://docs.python.org/3/tutorial/>
- [10] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- [11] Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics* (pp. 1715-1725). <https://doi.org/10.18653/v1/P16-1162>
- [12] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781. <https://arxiv.org/abs/1301.3781>
- [13] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019* (pp. 3982-3992). <https://doi.org/10.18653/v1/D19-1410>
- [14] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *3rd International Conference on Learning Representations (ICLR 2015)*. <https://arxiv.org/abs/1409.0473>
- [15] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171-4186). <https://doi.org/10.18653/v1/N19-1423>
- [16] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI.
- [17] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations (ICLR 2021)*. <https://arxiv.org/abs/2010.11929>
- [18] Mousavi, S. M., Ellsworth, W. L., Zhu, W., Chuang, L. Y., & Beroza, G. C. (2020). Earthquake transformer: An attentive deep-learning model for simultaneous earthquake detection and phase picking. *Nature Communications*, 11, 3952. <https://doi.org/10.1038/s41467-020-17591-w>
- [19] Kaplan, J., McCandlish, S., Henighan, T., et al. (2020). Scaling laws for neural language models. arXiv preprint arXiv:2001.08361. <https://arxiv.org/abs/2001.08361>
- [20] Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35* (pp. 24824-24837). <https://arxiv.org/abs/2201.11903>
- [21] Bommasani, R., Hudson, D. A., Adeli, E., et al. (2021). On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258. <https://arxiv.org/abs/2108.07258>
- [22] Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. In *2nd International Conference on Learning Representations (ICLR 2014)*. <https://arxiv.org/abs/1312.6114>
- [23] Goodfellow, I., Pouget-Abadie, J., Mirza, M., et al. (2014). Generative adversarial nets. In *Advances in Neural Information Processing Systems 27* (pp. 2672-2680). <https://arxiv.org/abs/1406.2661>
- [24] Merchant, A., Batzner, S., Schoenholz, S. S., et al. (2023). Scaling deep learning for materials discovery. *Nature*, 624(7990), 80-85. <https://doi.org/10.1038/s41586-023-06735-9>
- [25] Boiko, D. A., MacKnight, R., Kline, B., & Gomes, G. (2023). Autonomous chemical research with large language models. *Nature*, 624(7992), 570-578. <https://doi.org/10.1038/s41586-023-06792-0>

- [26] Kose, U. (2019). Techniques for adversarial examples threatening the safety of artificial intelligence based systems. In *I. International Science and Innovation Congress (INSI Congress 2019)*. Pamukkale, Denizli, Türkiye. <https://arxiv.org/abs/1910.06907>
- [27] Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 3645-3650). <https://doi.org/10.18653/v1/P19-1355>